SWE1, Session 7

Use Case Testing

Midterm evaluation

- Balance between theory and exercises is off
  - I'll turn down the theory in the future
- Too much repetition from last semester
  - I'll be more careful and keep it to a minimum
- The students don't learn too much new information
  - Hopefully, the above will fix that
- I'm sometimes difficult to hear
  - I'll try to speak up - please remind me if I forget

Why test?

- Suggestions from the class:
  - To avoid breakage
  - To test for faster options
  - To check if all the requirements are met
  - To fulfil the definition of done
- My addition:
  - To find bugs

Testing

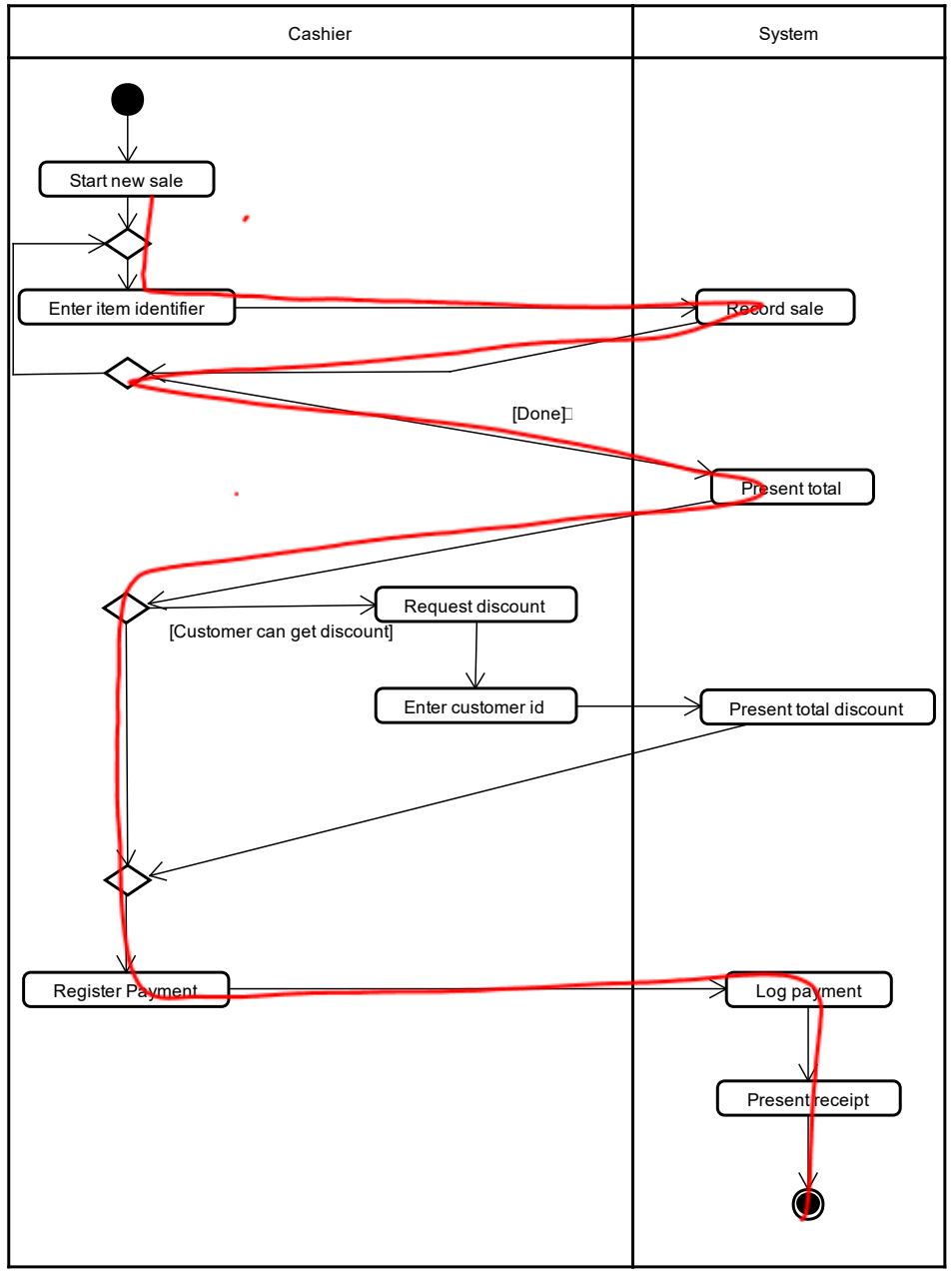
- We test to find errors
- Errors:
  - Deviations from the use cases / functional requirements
  - Deviations from the non- functional requirements
- Today, we focus on use cases
- Use case testing is *black box testing* - we treat the system as a black box

Use case scenarios

- Main scenario
  - aka. sunshine scenario, main sequence, base sequence, etc.
- Alternate scenarios
  - An error happens
  - The user makes a different decision
  - A special case applies
- Combined scenarios
  - Scenarios where more alternate scenarios combines

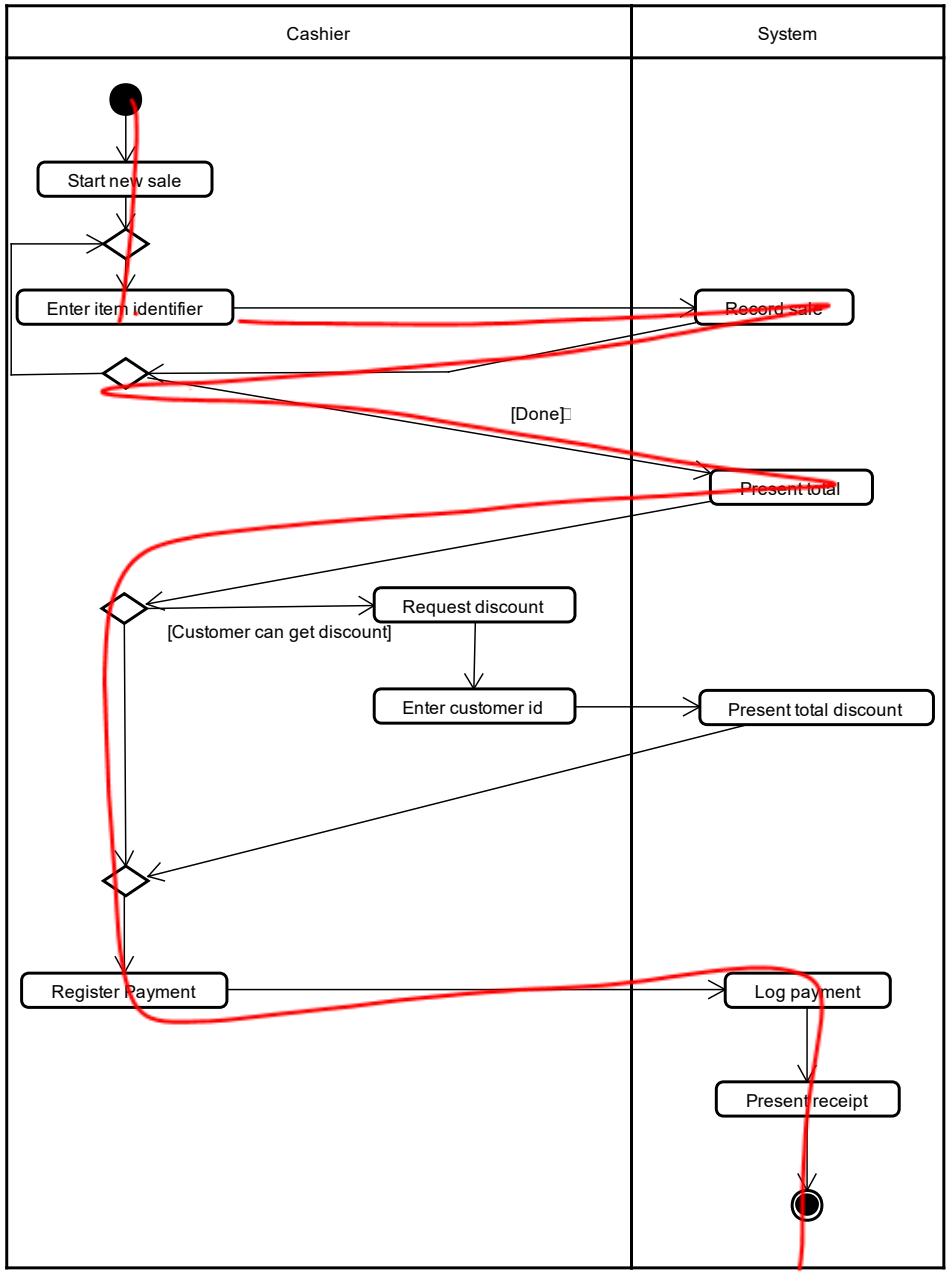
Finding scenarios

- Take them directly from the use case description
  - Harder than you think
- Create an activity diagram
  - More work
  - Better results



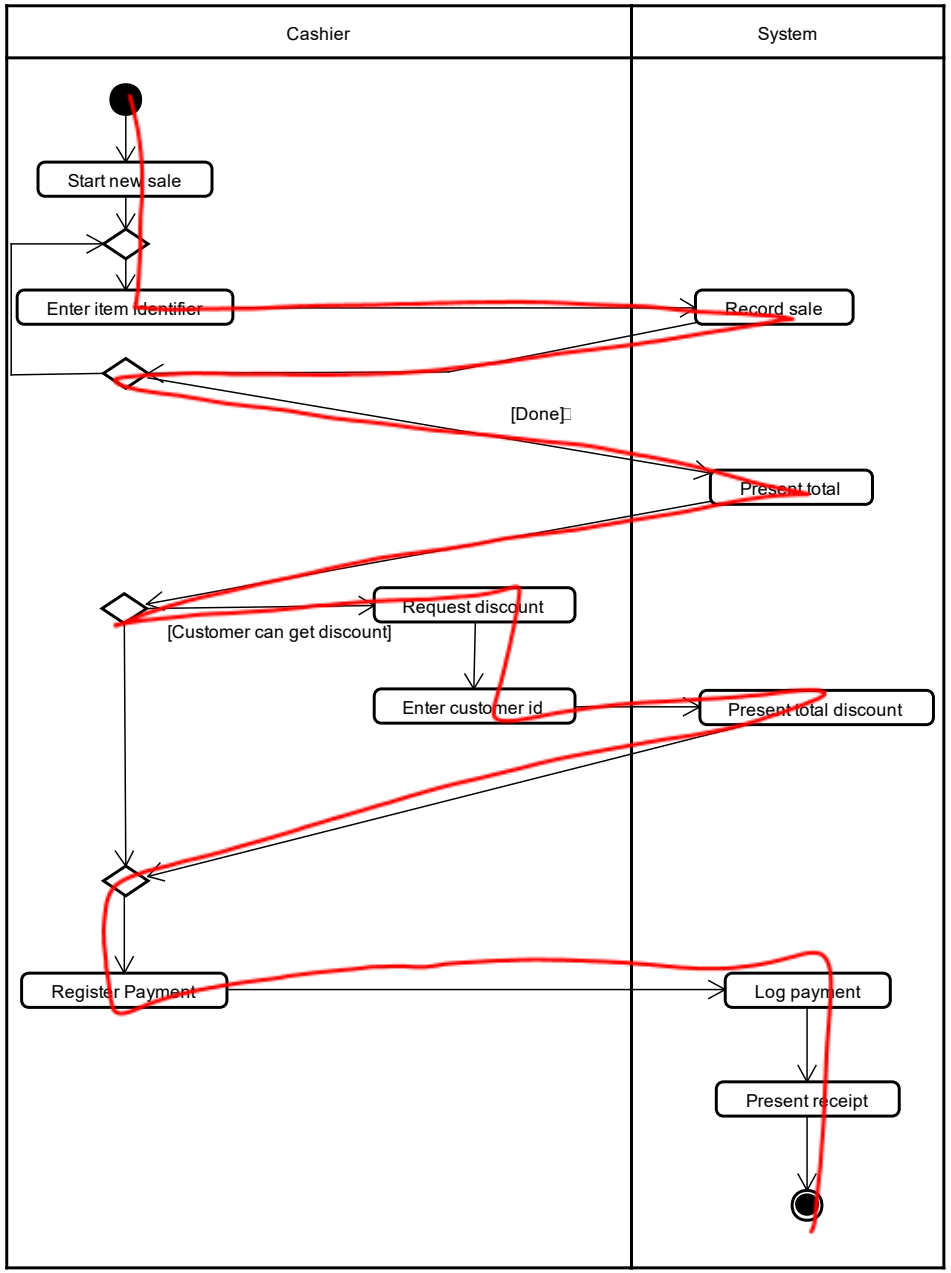
Main scenario

- The user purchases one item
- No discount



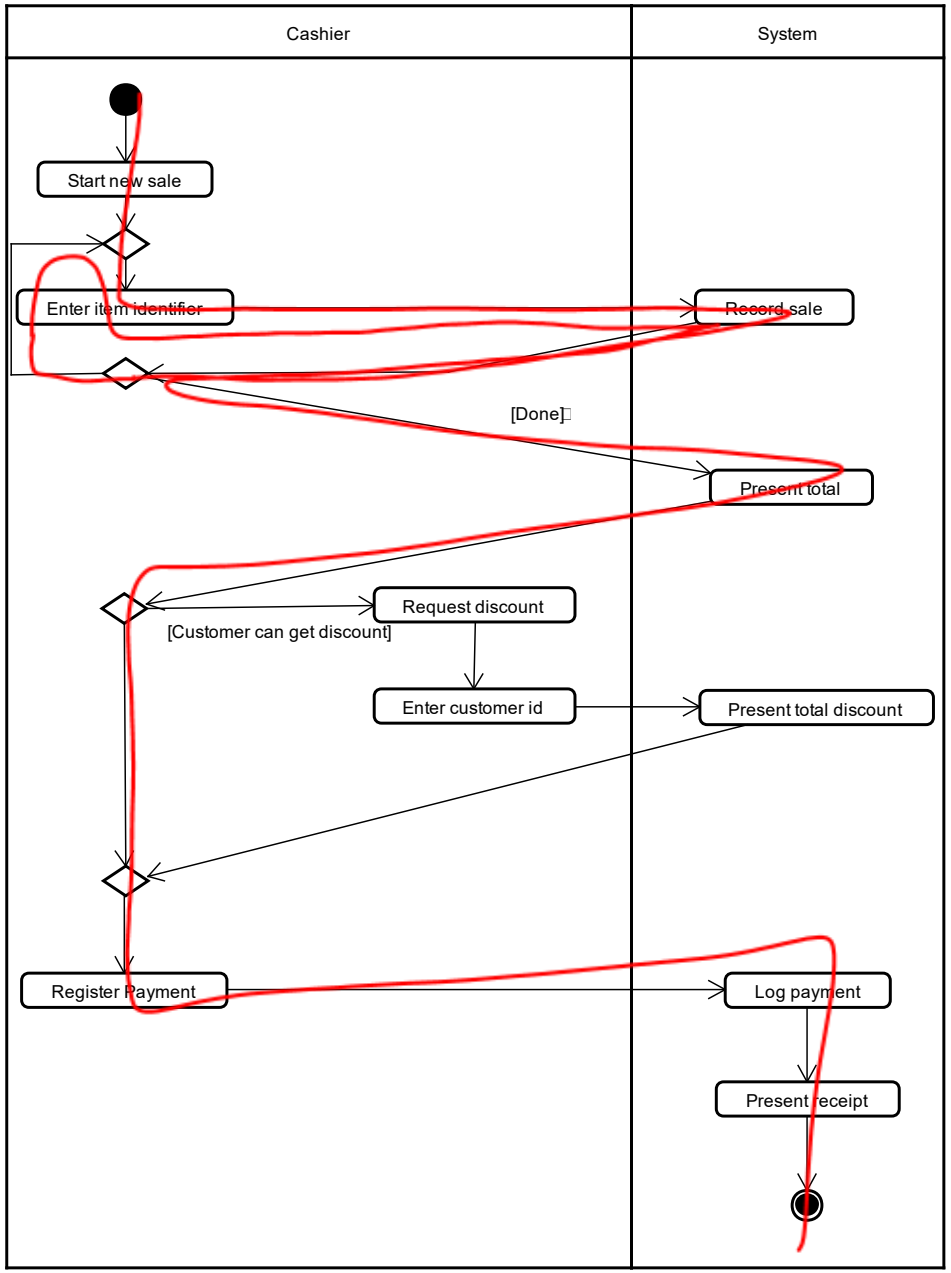
Alternate scenarios

- The user buys one item
- Apply discount



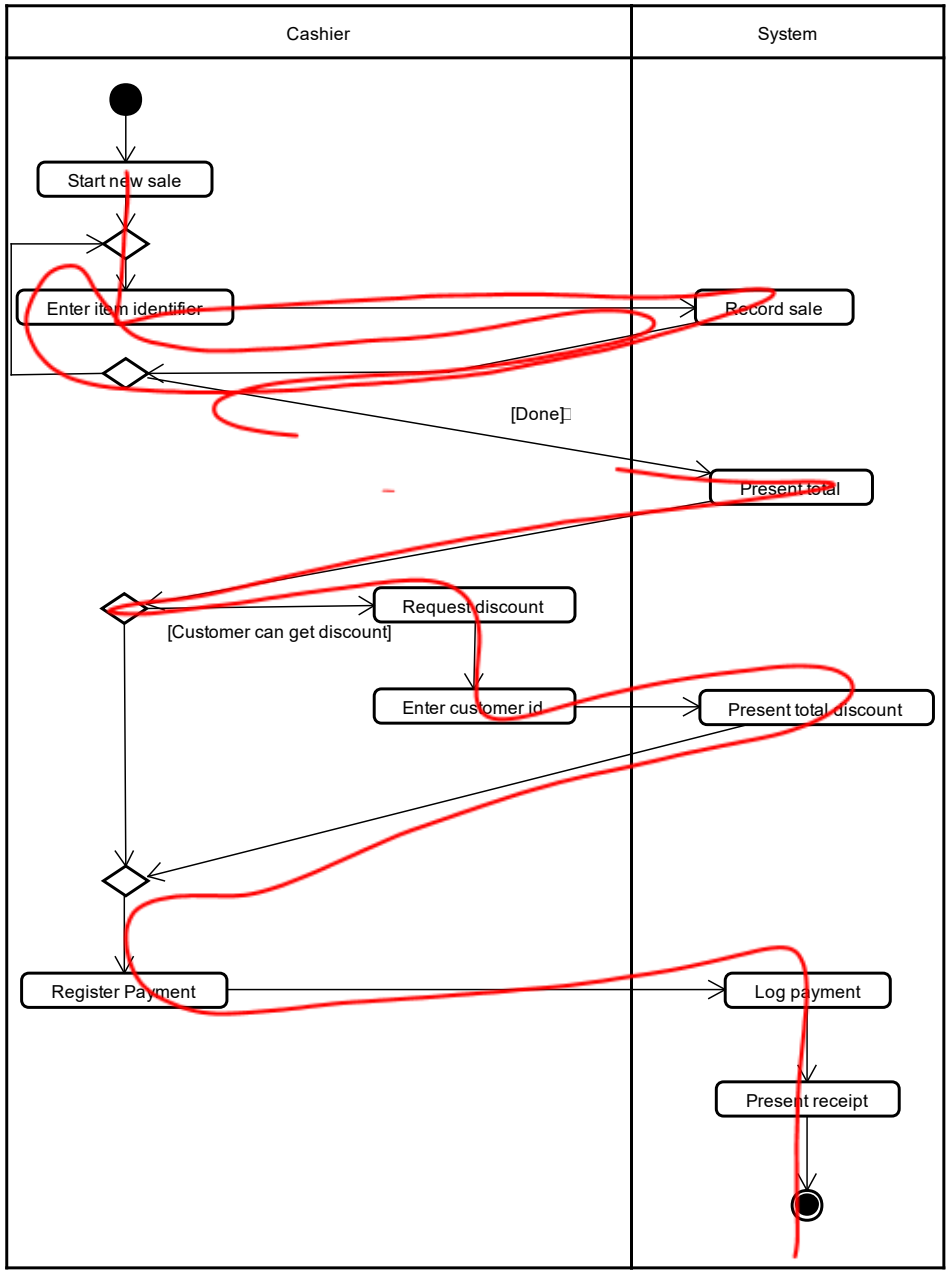
Main scenario (2)

- The user buys 2 items
- No discount



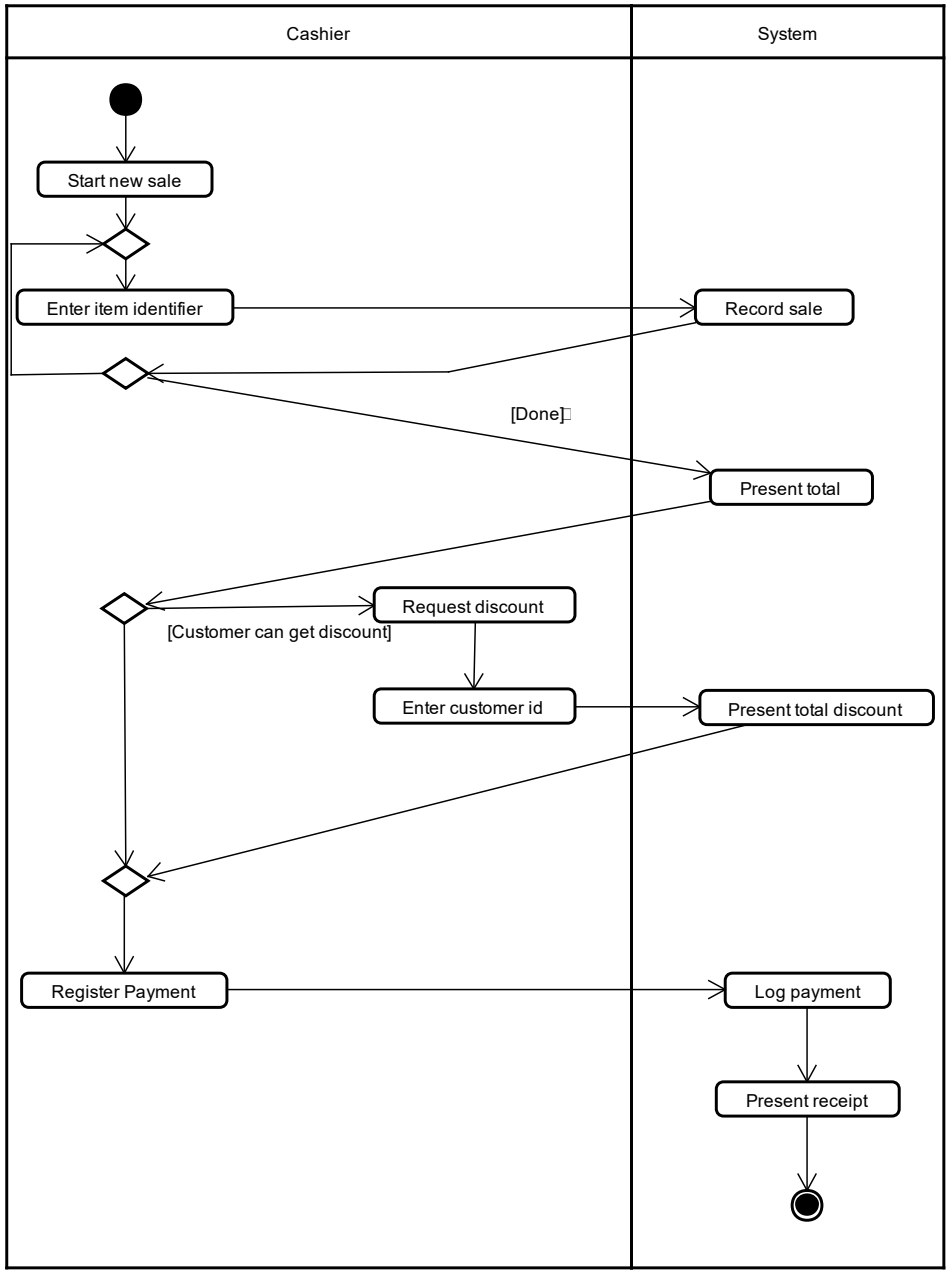
Compound scenario

- The user buys two items
- Apply discount



And more…

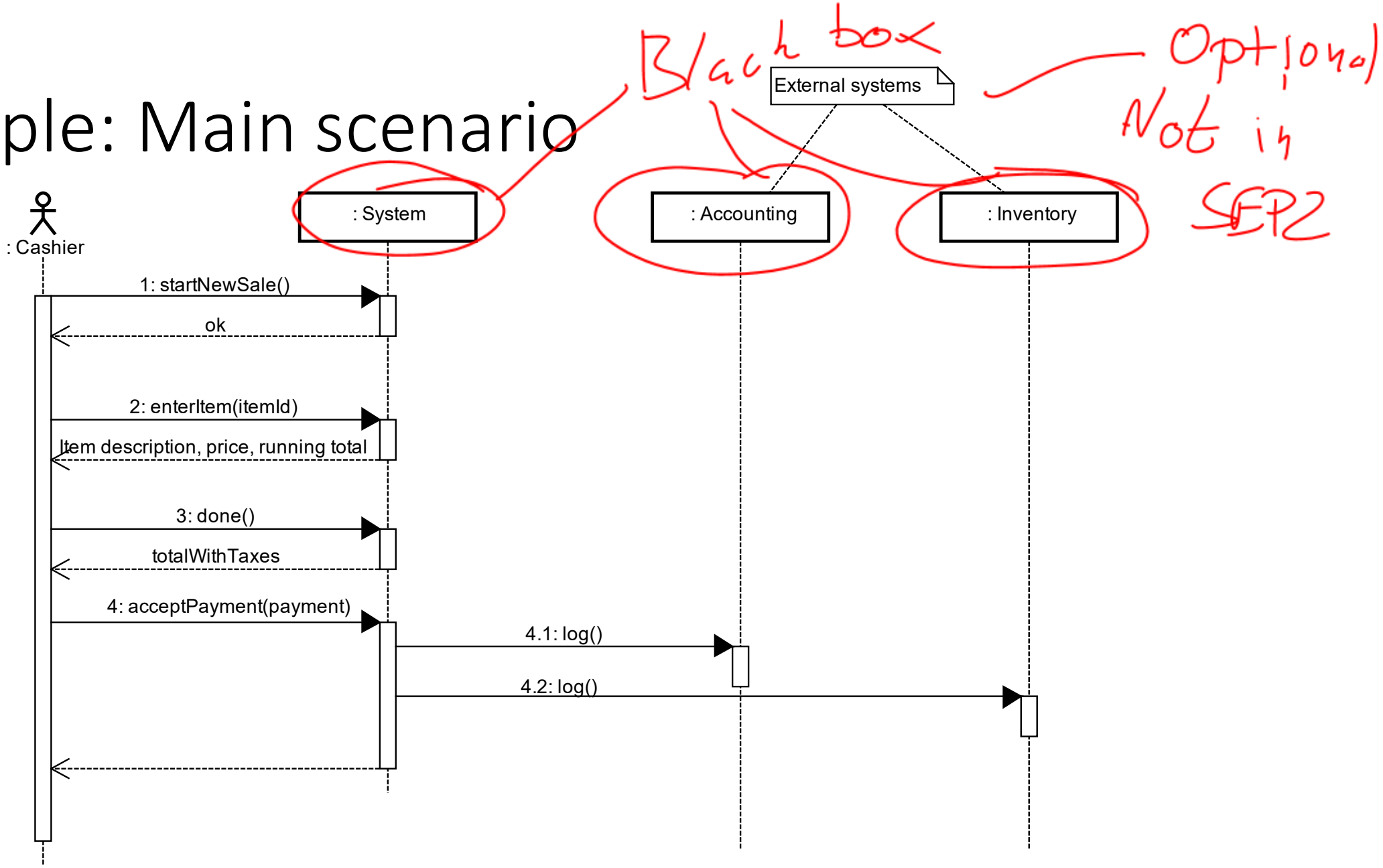
- This use case is missing
  - Can the user buy 0 items?
  - Discount doesn't apply to user.
  - Payment fails
  - And more



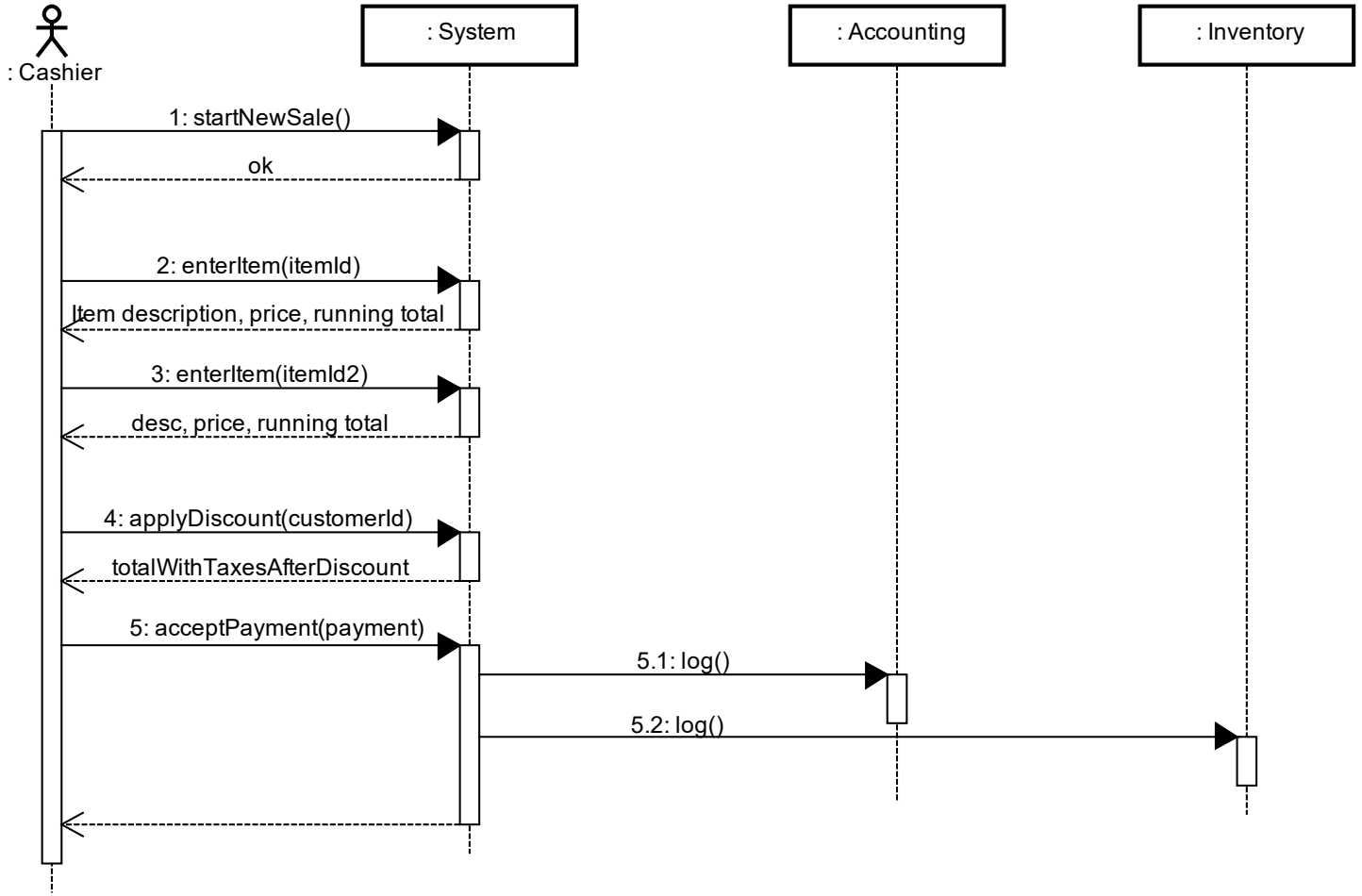
Documenting scenarios

- We use **System Sequence Diagrams** to document scenarios
  - One diagram per scenario
  - **No loops or branches (opt/alt)**
- A System Sequence Diagram:
  - A sequence diagram
  - Only has the actor and the system as lifelines
  - Alternatively, can have *external* systems as well
  - **No system details**

Example: Main scenario

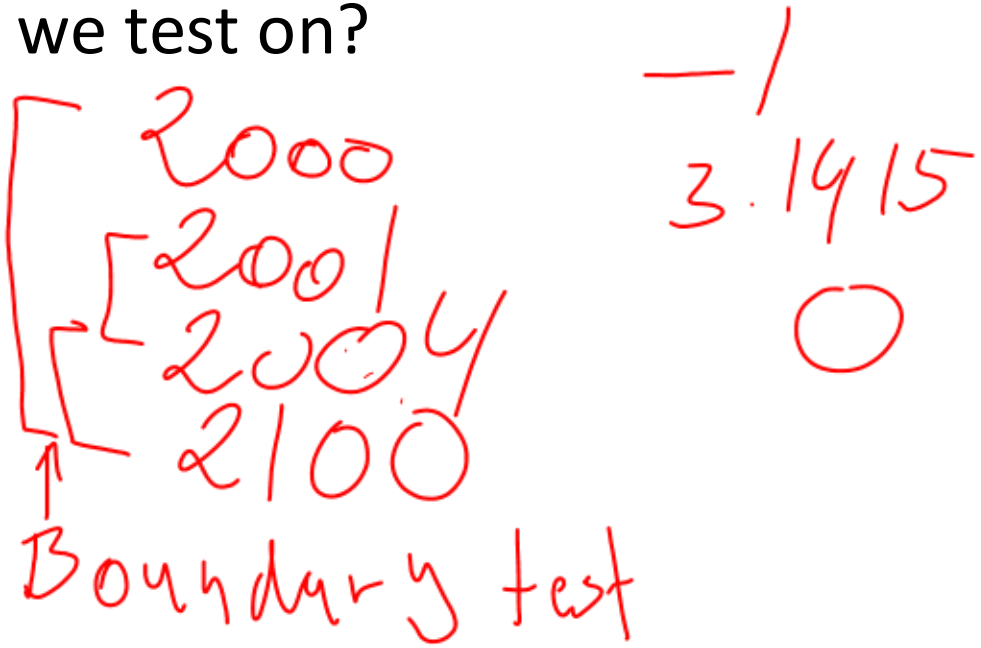


Example: Combined Scenario



Test data

- Look at the definition of leap year on the right
- Consider: Which years should we test on?



Every year that is exactly divisible by four is a leap year, except for years that are exactly divisible by 100, but these centurial years are leap years if they are exactly divisible by 400. For example, the years 1700, 1800, and 1900 are not leap years, but the years 1600 and 2000 are.

(Source: Wikipedia)

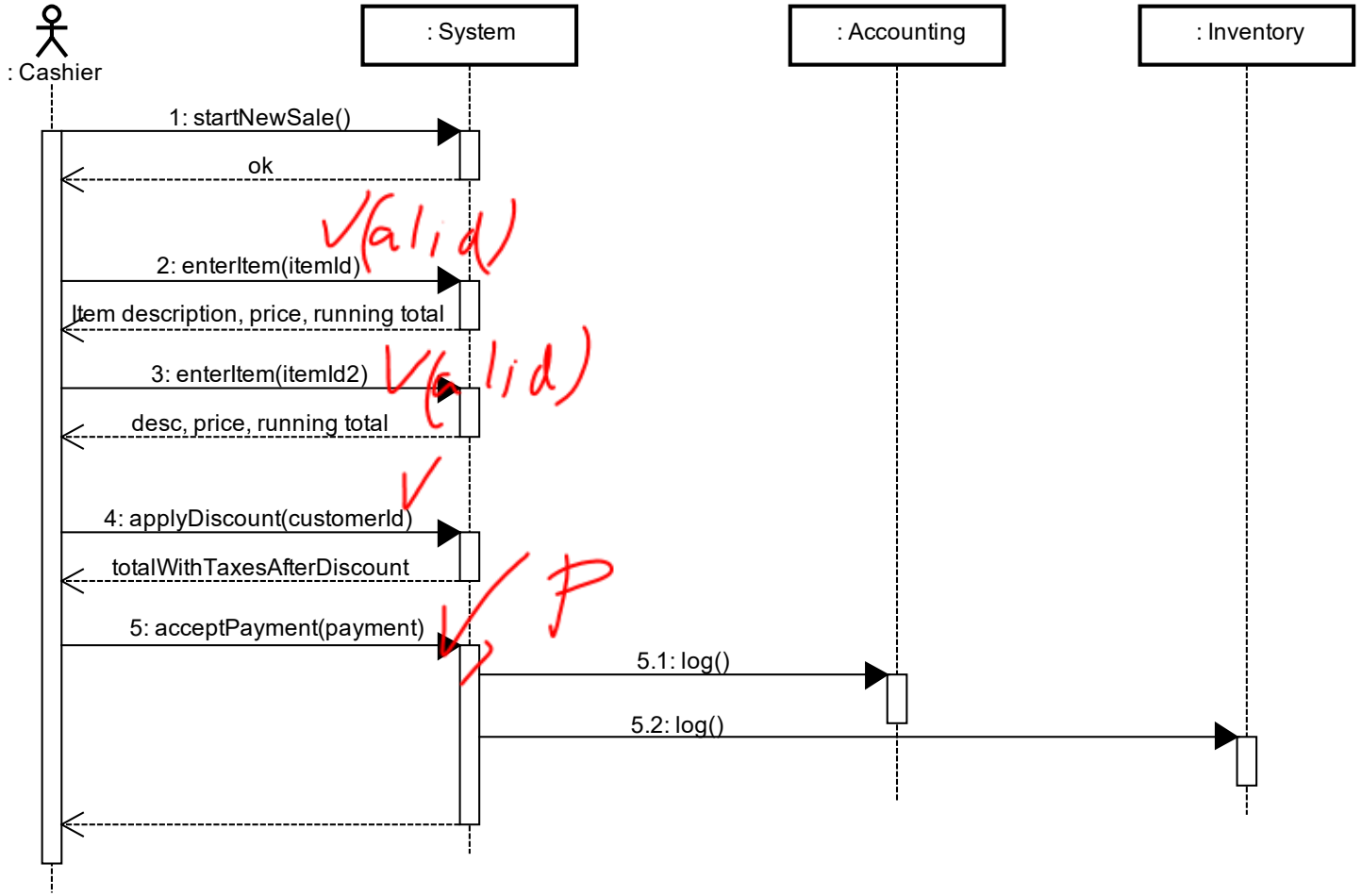
Test data

- Which *kind* of test data do we need?
  - Z – zero
  - O – One
  - M – Many
  - P – positive number
  - N – negative number
  - V – valid input (id, cpr, username, password, …)
  - I – invalid input
  - B – boundary (close to maximum, just over maximum)
  - E – empty (like empty string)

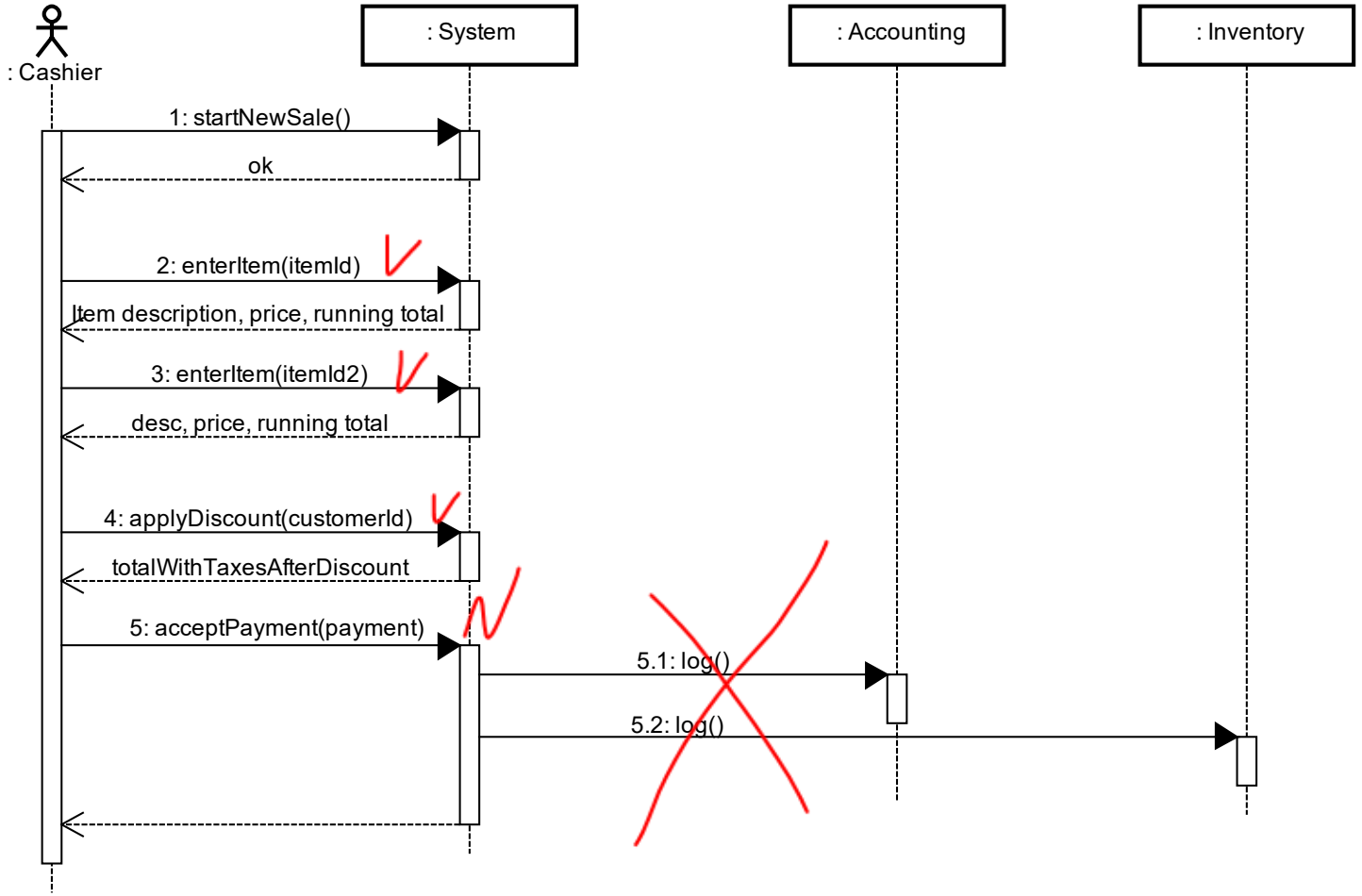
Things to consider

- Many scenarios require certain kinds of test data.
- Sometimes you need several test for one scenario

Test of Combined Scenario - happy path



Test of Combined Scenario - bad payment



Test case preconditions

- Items exist:
  - Id: 63245, price: $2.34
  - Id: 15487, price: $4.56
- CustomerId exist: 5467
- Customer is eligible for 10% discount
- CardId: 4571 6589 1234 5678 is valid

Test case

Step	Test Steps	Test Data	Expected Result
1	Start new sale		Sales page displayed
2	Enter Item	63245	-
3	Enter Item	15481	-
4	Press “Done”		Total is $6.90
5	Press “Apply Discount”		System prompts for customerId
6	Enter customerId	5467	Discount is $0.69 Total is $6.21
7	Enter Payment	$6.21	Success